

Pre Evaluacion FEIP 2026

Ejercicios unicos reordenados por tema - codigo destacado en bloques grises

Criterio: no se conserva la numeracion original FEIP porque habia ejercicios repetidos. La numeracion nueva es correlativa y los ejercicios se agrupan por tema.

Indice

BLOQUE 1 - Arrays, arreglos y matrices

Ejercicio 1 - Mostrar arreglo completo con Arrays.toString

Ejercicio 2 - Arreglo de objetos Libro

Ejercicio 3 - Padovan

Ejercicio 4 - Operaciones con matrices

Ejercicio 5 - Suma por filas en matriz escalonada

Ejercicio 6 - Promedio de números pares

BLOQUE 2 - String y fechas

Ejercicio 7 - Comparación de String

Ejercicio 8 - substring

Ejercicio 9 - Crear LocalDate

Ejercicio 10 - Excepciones con String

Ejercicio 11 - Validar vocales

BLOQUE 3 - Metodos, variables y operadores

Ejercicio 12 - Método sin retorno

Ejercicio 13 - Procedimiento void

Ejercicio 14 - Tipo de retorno

Ejercicio 15 - Nombres de variables

Ejercicio 16 - Operadores y ternario

Ejercicio 17 - Booleanos con ternario

BLOQUE 4 - POO, clases y objetos

Ejercicio 18 - Objeto o instancia

Ejercicio 19 - Clase Libro completa

Ejercicio 20 - Clase abstracta

Ejercicio 21 - Clase abstracta Auto y polimorfismo

Ejercicio 22 - Cuentas bancarias

BLOQUE 5 - Sobrecarga

Ejercicio 23 - Sobrecarga con short

Ejercicio 24 - Sobrecarga válida

BLOQUE 6 - Colecciones

Ejercicio 25 - HashMap

Ejercicio 26 - LinkedHashSet

Ejercicio 27 - Queue con offer y poll

Ejercicio 28 - CRUD de reservas con HashMap

BLOQUE 7 - Excepciones

Ejercicio 29 - Afirmaciones sobre excepciones

Ejercicio 30 - Flujo con try, catch, finally

Ejercicio 31 - Excepción no controlada en try interno

Ejercicio 32 - Afirmaciones incorrectas sobre excepciones

Ejercicio 33 - Flujo sin excepción

BLOQUE 1 - Arrays, arreglos y matrices

Ejercicio 1 - Mostrar arreglo completo con Arrays.toString

CODIGO + EXPLICACION COMENTADA

```
import java.util.Arrays;

public class Principal {

    public static void main(String[] args) {

        // Creamos un arreglo de enteros.
        int[] numeros = {10, 20, 30, 40};

        // Si imprimimos directamente el arreglo:
        //
        // System.out.println(numeros);
        //
        // Java NO muestra el contenido.
        // Muestra algo parecido a una referencia de memoria.
        //
        // Para mostrar el contenido completo del arreglo
        // en formato [10, 20, 30, 40],
        // usamos Arrays.toString().
        System.out.println(Arrays.toString(numeros));
    }
}

/*
RESPUESTA CORRECTA:
a. System.out.println(Arrays.toString(numeros));

SALIDA:
[10, 20, 30, 40]

IDEA CLAVE:

Para arreglos:
Arrays.toString(arreglo)

Para listas:
lista.toString()

No confundas:

System.out.println(numeros);

Eso no imprime los valores del arreglo como querés.
Java ahí se pone misterioso, como Git cuando dice "detached HEAD".
*/
```

Ejercicio 2 - Arreglo de objetos Libro

CODIGO + EXPLICACION COMENTADA

```
public class Array {

    public static void main(String[] args) {

        // Creamos un arreglo de objetos Libro con 3 posiciones.
        Libro libros[] = new Libro[3];

        // Posición 0:
        // Primero se guarda "SQL".
        libros[0] = new Libro("SQL");

        // Posición 1:
        // Primero se guarda "Python".
        libros[1] = new Libro("Python");

        // Posición 2:
        // Primero se guarda "C#".
        libros[2] = new Libro("C#");

        // ATENCIÓN:
        // Ahora volvemos a escribir en la posición 1.
        //
        // Antes:
```

```

// libros[1] = "Python"
//
// Después:
// libros[1] = "HTML"
//
// Python se pierde porque fue reemplazado.
libros[1] = new Libro("HTML");

// ATENCIÓN:
// Ahora volvemos a escribir en la posición 0.
//
// Antes:
// libros[0] = "SQL"
//
// Después:
// libros[0] = "Java"
//
// SQL se pierde porque fue reemplazado.
libros[0] = new Libro("Java");

// Recorremos el arreglo con for-each.
//
// En cada vuelta, l representa un Libro del arreglo.
for (Libro l : libros) {

// Imprimimos el título del libro actual.
System.out.print(l.getTitulo() + " ");
}
}
}

/*
ESTADO FINAL DEL ARREGLO:

libros[0] = Java
libros[1] = HTML
libros[2] = C#

SALIDA:
Java HTML C#

RESPUESTA:
d. Java HTML C#

IDEA CLAVE:

Cuando asignás dos veces en la misma posición,
el valor anterior se reemplaza.

No se agrega.
No se corre.
Se pisa.

libros[1] = "Python";
libros[1] = "HTML";

Al final queda:
HTML

Python quedó como recuerdo académico.
*/

```

Ejercicio 3 - Padovan

CODIGO + EXPLICACION COMENTADA

```

public int[] padovan(int n) {

// Si n es 0 o negativo,
// no hay términos para generar.
//
// Se devuelve un arreglo vacío.
if (n <= 0) {
return new int[0];
}

// Creamos un arreglo de tamaño n.
//
// Si n = 10, el arreglo tiene 10 posiciones:
// 0 hasta 9.
int[] serie = new int[n];

```

```
// Recorremos todas las posiciones del arreglo.
for (int i = 0; i < n; i++) {

// Los tres primeros valores de Padovan son:
//
// P(0) = 1
// P(1) = 1
// P(2) = 1
//
// Por eso si i es 0, 1 o 2,
// guardamos 1.
if (i <= 2) {

serie[i] = 1;

} else {

// Para los demás términos:
//
// P(n) = P(n - 2) + P(n - 3)
//
// Ejemplo:
// P(3) = P(1) + P(0) = 1 + 1 = 2
//
// P(4) = P(2) + P(1) = 1 + 1 = 2
//
// P(5) = P(3) + P(2) = 2 + 1 = 3
serie[i] = serie[i - 2] + serie[i - 3];
}
}

// Devolvemos el arreglo completo.
return serie;
}

/*
EJEMPLO:

padovan(10)

Posiciones:

0 -> 1
1 -> 1
2 -> 1
3 -> serie[1] + serie[0] = 1 + 1 = 2
4 -> serie[2] + serie[1] = 1 + 1 = 2
5 -> serie[3] + serie[2] = 2 + 1 = 3
6 -> serie[4] + serie[3] = 2 + 2 = 4
7 -> serie[5] + serie[4] = 3 + 2 = 5
8 -> serie[6] + serie[5] = 4 + 3 = 7
9 -> serie[7] + serie[6] = 5 + 4 = 9

RESULTADO:
[1, 1, 1, 2, 2, 3, 4, 5, 7, 9]

IDEA CLAVE:

Los primeros tres valores son base.
Después cada valor depende de dos posiciones anteriores:
i - 2
i - 3
*/
```

Ejercicio 4 - Operaciones con matrices

CODIGO + EXPLICACION COMENTADA

```
public class OperacionesConMatrices {

public int sumaElementosMatriz(int[][] matriz) {

// Acumulador para guardar la suma total.
int suma = 0;

// Primer for:
// recorre filas.
for (int i = 0; i < matriz.length; i++) {

// Segundo for:
```

```

// recorre columnas de la fila actual.
for (int j = 0; j < matriz[i].length; j++) {

    // Sumamos cada elemento.
    suma += matriz[i][j];
}
}

// Devolvemos la suma total.
return suma;
}

public int[][] sumarMatrices(int[][] matriz1, int[][] matriz2) {

    // Creamos una matriz resultado
    // con la misma cantidad de filas.
    int[][] resultado = new int[matriz1.length][];

    // Recorremos las filas.
    for (int i = 0; i < matriz1.length; i++) {

        // Cada fila del resultado tendrá
        // el mismo largo que la fila correspondiente.
        resultado[i] = new int[matriz1[i].length];

        // Recorremos las columnas.
        for (int j = 0; j < matriz1[i].length; j++) {

            // Sumamos elemento a elemento.
            //
            // resultado[i][j] =
            // matriz1[i][j] + matriz2[i][j]
            resultado[i][j] = matriz1[i][j] + matriz2[i][j];
        }
    }

    // Devolvemos la matriz sumada.
    return resultado;
}

public void imprimirMatriz(int[][] matriz) {

    // Recorremos las filas.
    for (int i = 0; i < matriz.length; i++) {

        // Recorremos las columnas.
        for (int j = 0; j < matriz[i].length; j++) {

            // Imprimimos el elemento y un espacio.
            System.out.print(matriz[i][j] + " ");
        }

        // Al terminar una fila, bajamos de línea.
        System.out.println();
    }
}

/*
IDEA CLAVE:

sumaElementosMatriz:
devuelve un int con la suma total.

sumarMatrices:
devuelve una nueva matriz.

imprimirMatriz:
es void porque solo imprime.

Matriz = for adentro de for.

i -> fila
j -> columna

Si esto te queda claro, las matrices dejan de parecer un tablero del terror.
*/

```

Ejercicio 5 - Suma por filas en matriz escalonada

CODIGO + EXPLICACION COMENTADA

```
public static void mostrarSumaFilas(int[][] matriz) {

    // Recorremos cada fila de la matriz.
    for (int i = 0; i < matriz.length; i++) {

        // Acumulador para la fila actual.
        int suma = 0;

        // Recorremos los elementos de la fila i.
        //
        // Usamos matriz[i].length porque la matriz
        // puede ser escalonada.
        //
        // Eso significa que cada fila puede tener distinto largo.
        for (int j = 0; j < matriz[i].length; j++) {

            // Sumamos cada elemento de la fila.
            suma += matriz[i][j];
        }

        // La numeración de filas empieza en 1,
        // pero el índice i empieza en 0.
        //
        // Por eso imprimimos i + 1.
        System.out.println("Fila " + (i + 1) + ": " + suma);
    }
}

/*
EJEMPLO:

int[][] matriz = {
    {1, 2, 3},
    {4, 5},
    {},
    {10}
};

Fila 1:
1 + 2 + 3 = 6

Fila 2:
4 + 5 = 9

Fila 3:
no tiene elementos -> suma 0

Fila 4:
10

SALIDA:

Fila 1: 6
Fila 2: 9
Fila 3: 0
Fila 4: 10

IDEA CLAVE:

Matriz escalonada:
las filas pueden tener largos distintos.

Por eso nunca usamos matriz[0].length para todas.
Usamos:

matriz[i].length

Eso evita errores.
*/
```

Ejercicio 6 - Promedio de números pares

CODIGO + EXPLICACION COMENTADA

```
public static double calcularPromedioPares(int[] array) {

    // Acumulador para sumar solo los pares.
    int suma = 0;
```

```
// Contador para saber cuántos pares encontramos.
int contador = 0;

// Recorremos el arreglo completo.
for (int i = 0; i < array.length; i++) {

    // Verificamos si el número es par.
    //
    // Un número es par si el resto de dividir entre 2 es 0.
    if (array[i] % 2 == 0) {

        // Sumamos el número par.
        suma += array[i];

        // Contamos un par más.
        contador++;
    }
}

// Si no hay pares, no podemos dividir entre 0.
//
// El enunciado pide devolver 0.
if (contador == 0) {
    return 0;
}

// Promedio:
//
// suma de pares / cantidad de pares.
//
// Usamos (double) para asegurar resultado decimal.
return (double) suma / contador;
}

/*
EJEMPLO 1:

int[] numeros = {2, 4, 6, 8};

Pares:
2, 4, 6, 8

Suma:
20

Cantidad:
4

Promedio:
20 / 4 = 5.0

EJEMPLO 2:

int[] numeros = {1, 3, 5, 7};

No hay pares.

contador = 0

Resultado:
0.0

IDEA CLAVE:

Para promedio necesitas dos cosas:

1) suma
2) cantidad

Y si cantidad es 0,
no dividís.

Dividir entre cero es mala idea matemática y peor idea en Java.
*/
```

BLOQUE 2 - String y fechas

Ejercicio 7 - Comparación de String

CODIGO + EXPLICACION COMENTADA

```
public class PruebaString {

    public static void main(String[] args) {

        // x e y se crean como literales.
        //
        // En Java, los literales iguales suelen compartir
        // la misma referencia en el String pool.
        String x = "Java";
        String y = "Java";

        // z se crea con new.
        //
        // Aunque el contenido sea "Java",
        // se crea otro objeto distinto en memoria.
        String z = new String("Java");

        // x == y compara referencias.
        //
        // Como ambos vienen del String pool,
        // normalmente apuntan al mismo objeto.
        //
        // Resultado:
        // true
        System.out.print(x == y);

        System.out.print(" ");

        // x == z compara referencias.
        //
        // z fue creado con new,
        // entonces no es la misma referencia que x.
        //
        // Resultado:
        // false
        System.out.print(x == z);

        System.out.print(" ");

        // equals compara contenido.
        //
        // x contiene "Java"
        // z contiene "Java"
        //
        // Resultado:
        // true
        System.out.print(x.equals(z));
    }
}

/*
SALIDA:
true false true

RESPUESTA:
d. true false true

IDEA CLAVE:

== compara referencia de memoria.

equals() compara contenido.

Para String, casi siempre usá equals().
El == con String es una trampa clásica.
Clásica como olvidarse el break en un switch.
*/
```

Ejercicio 8 - substring

CODIGO + EXPLICACION COMENTADA

```
public class Principal {
```



```

public static void main(String[] args) {

String texto = "Programacion";

// substring(0, 4) toma caracteres desde índice 0
// hasta índice 4, pero SIN incluir el índice 4.
//
// Texto:
//
// P r o g r a m a c i o n
// 0 1 2 3 4 5 6 7 8 9 10 11
//
// substring(0, 4)
// toma índices:
//
// 0, 1, 2, 3
//
// Eso forma:
// Prog
System.out.println(texto.substring(0, 4));
}
}

/*
SALIDA:
Prog

RESPUESTA:
a. Prog

IDEA CLAVE:

substring(inicio, fin)

Incluye:
inicio

No incluye:
fin

Por eso substring(0, 4) no toma la letra del índice 4.
*/

```

Ejercicio 9 - Crear LocalDate

CODIGO + EXPLICACION COMENTADA

```

import java.time.LocalDate;

public class Fechas {

public static void main(String[] args) {

// Forma correcta para crear un LocalDate:
//
// LocalDate.of(año, mes, día)
LocalDate fecha = LocalDate.of(2026, 04, 30);

System.out.println(fecha);
}
}

/*
RESPUESTA:
b. LocalDate fecha = LocalDate.of(2026, 04, 30)

POR QUÉ:

LocalDate.create(...)
no existe.

Date.of(...)
no corresponde a LocalDate.

new LocalDate(...)
no se usa porque LocalDate no se instancia así.

IDEA CLAVE:

Para LocalDate:

```

```
LocalDate.of(año, mes, día)
```

Ejemplo:

```
LocalDate.of(2026, 4, 30)
```

```
*/
```

Ejercicio 10 - Excepciones con String

CODIGO + EXPLICACION COMENTADA

```
public void prueba(String dato) {

    try {

        // charAt(3) intenta acceder al carácter en la posición 3.
        //
        // Si dato es null:
        // NullPointerException.
        //
        // Si dato tiene menos de 4 caracteres:
        // StringIndexOutOfBoundsException.
        System.out.println(dato.charAt(3));

        // substring(0, 5) intenta tomar desde índice 0 hasta 5,
        // sin incluir 5.
        //
        // Necesita que el String tenga al menos 5 caracteres.
        System.out.println(dato.substring(0, 5));

        // Solo se imprime si las dos instrucciones anteriores
        // funcionaron correctamente.
        System.out.println("Proceso correcto");

    } catch (NullPointerException e) {

        // Si dato es null,
        // no se puede usar charAt ni substring.
        System.out.println("String null");

    } catch (StringIndexOutOfBoundsException e) {

        // Si el índice no existe,
        // se captura esta excepción.
        System.out.println("Índice fuera de rango");

    } catch (Exception e) {

        // Cualquier otro error no previsto.
        System.out.println("Error general");
    }
}

/*
EJEMPLO 1:

dato = null

dato.charAt(3) falla con NullPointerException.

Imprime:
String null

EJEMPLO 2:

dato = "abc"

charAt(3) falla porque solo hay índices 0, 1, 2.

Imprime:
Índice fuera de rango

EJEMPLO 3:

dato = "Programacion"

charAt(3) imprime:
g

substring(0, 5) imprime:
Progr
```

Luego:
Proceso correcto

IDEA CLAVE:

NullPointerException:
cuando el objeto no existe.

StringIndexOutOfBoundsException:
cuando el índice no existe.

Exception:
catch general.

Siempre poner primero los catch específicos
y al final el catch general.
*/

Ejercicio 11 - Validar vocales

CODIGO + EXPLICACION COMENTADA

```
public boolean validarVocal(String palabra) {  
  
    // Primero validamos el largo.  
    //  
    // El largo debe estar entre 3 y 6 inclusive.  
    //  
    // Si tiene menos de 3 o más de 6,  
    // directamente no es válida.  
    if (palabra.length() < 3 || palabra.length() > 6) {  
        return false;  
    }  
  
    // Recorremos cada carácter del String.  
    for (int i = 0; i < palabra.length(); i++) {  
  
        // Obtenemos la letra actual.  
        char letra = palabra.charAt(i);  
  
        // Verificamos que sea una vocal válida.  
        //  
        // Se aceptan:  
        // a e i o u  
        // A E I O U  
        //  
        // No se aceptan tildes:  
        // á, é, í, ó, ú  
        //  
        // No se aceptan consonantes.  
        // No se aceptan símbolos.  
        if (letra != 'a' && letra != 'e' && letra != 'i' &&  
            letra != 'o' && letra != 'u' &&  
            letra != 'A' && letra != 'E' && letra != 'I' &&  
            letra != 'O' && letra != 'U') {  
  
            // Si una sola letra no es vocal válida,  
            // toda la palabra queda inválida.  
            return false;  
        }  
    }  
  
    // Si pasó el largo  
    // y todas las letras eran vocales,  
    // la palabra es válida.  
    return true;  
}  
  
/*  
EJEMPLO 1:  
  
"Aei"  
  
Largo:  
3 -> válido.  
  
Letras:  
A -> vocal  
e -> vocal  
i -> vocal
```

Resultado:
true

EJEMPLO 2:

"aeijk"

Largo:
5 -> válido.

Letras:
a -> vocal
e -> vocal
i -> vocal
j -> NO es vocal

Resultado:
false

EJEMPLO 3:

"aeiAEI"

Largo:
6 -> válido.

Todas son vocales.

Resultado:
true

EJEMPLO 4:

"Aáaui"

La á con tilde NO está contemplada.

Resultado:
false

IDEA CLAVE:

Se deben cumplir dos condiciones:

- 1) largo entre 3 y 6
- 2) todas las letras deben ser vocales sin tilde

Si falla una condición:
return false.
*/

BLOQUE 3 - Metodos, variables y operadores

Ejercicio 12 - Método sin retorno

CODIGO + EXPLICACION COMENTADA

```
public class Ejemplo {  
  
    // Método sin retorno.  
    //  
    // La palabra clave es void.  
    //  
    // void significa:  
    // este método no devuelve ningún valor.  
    public void suma() {  
  
        // Puede hacer acciones.  
        // Por ejemplo imprimir, modificar atributos, etc.  
        System.out.println("Sumando...");  
    }  
}  
  
/*  
RESPUESTA CORRECTA:  
b. public void suma() { }  
  
POR QUÉ LAS OTRAS ESTÁN MAL:  
  
a. public suma() { }  
Falta el tipo de retorno.  
  
c. void public suma() { }  
El orden está mal.  
Primero va public, después void.  
  
d. public int suma(void) { }  
En Java no se usa void como parámetro.  
Además int indica que debería devolver un entero.  
  
IDEA CLAVE:  
  
Estructura correcta:  
  
modificador tipoRetorno nombreMetodo() {  
    // cuerpo  
}  
  
Ejemplo:  
  
public void suma() {  
}  
*/
```

Ejercicio 13 - Procedimiento void

CODIGO + EXPLICACION COMENTADA

```
public static void mostrarMensaje() {  
  
    // Este método no devuelve nada.  
    //  
    // Lo sabemos por la palabra void.  
    System.out.println("Hola");  
}  
  
/*  
RESPUESTA:  
a. Como un procedimiento, porque no devuelve valor.  
  
POR QUÉ:  
  
public:  
modificador de acceso.  
  
static:  
permite llamar al método sin crear objeto.  
  
void:  
indica que no devuelve valor.
```

mostrarMensaje:
nombre del método.

IDEA CLAVE:

Si tiene void:
procedimiento.

Si devuelve algo:
función.

Ejemplo función:

```
public static int sumar() {  
    return 5;  
}  
*/
```

Ejercicio 14 - Tipo de retorno

CODIGO + EXPLICACION COMENTADA

```
public static boolean esPar(int numero) {  
  
    // boolean es el tipo de retorno.  
    //  
    // Eso significa que este método devuelve:  
    //  
    // true  
    // o  
    // false  
    return numero % 2 == 0;  
}
```

/*

FIRMA:

```
public static boolean esPar(int numero)
```

public:
modificador de acceso.

static:
se puede llamar sin crear objeto.

boolean:
tipo de retorno.

esPar:
nombre del método.

int numero:
parámetro.

RESPUESTA:
d. El tipo de retorno

IDEA CLAVE:

Lo que aparece antes del nombre del método
es el tipo de retorno.

Ejemplos:

```
public int sumar()  
    retorna int.
```

```
public String getNombre()  
    retorna String.
```

```
public boolean esPar()  
    retorna boolean.
```

```
public void imprimir()  
    no retorna nada.  
*/
```

Ejercicio 15 - Nombres de variables

CODIGO + EXPLICACION COMENTADA

```
public class Variables {

    public static void main(String[] args) {

        // Correcto:
        int edad = 42;

        // Correcto:
        int edadAlumno = 20;

        // Correcto:
        int edad_alumno = 20;

        // Correcto, aunque no es lo más usado:
        int $precio = 100;

        // Incorrecto:
        //
        // int class = 5;
        //
        // class es palabra reservada.

        // Incorrecto:
        //
        // int 1numero = 10;
        //
        // No puede comenzar con número.

        // Incorrecto:
        //
        // int mi-variable = 5;
        //
        // No puede tener guion medio.
    }
}

/*
RESPUESTA:
a. No pueden usar palabras reservadas del lenguaje.

IDEA CLAVE:

Una variable en Java:

Puede tener:
letras
números
-
$

No puede:
empezar con número
usar palabras reservadas
tener guion medio
tener espacios

Ejemplos válidos:
nombre
nombreAlumno
nombre_alumno
precio1

Ejemplos inválidos:
1precio
class
mi-variable
mi variable
*/
```

Ejercicio 16 - Operadores y ternario

CODIGO + EXPLICACION COMENTADA

```
public class Operador {

    public static void main(String[] args) {

        int a = 10, b = 4, c = 7;
```

```

// a-- usa el valor actual y después resta 1.
//
// a pasa de 10 a 9.
a--;

// b += 3 significa:
// b = b + 3
//
// b pasa de 4 a 7.
b += 3;

// c-- hace que c pase de 7 a 6.
c--;

// For:
//
// i = 1
// i <= 2
// i++
//
// Se ejecuta dos veces: i = 1 e i = 2.
for (int i = 1; i <= 2; i++) {

// a -= 2 significa:
// a = a - 2
//
// Primera vuelta:
// a = 9 - 2 = 7
//
// Segunda vuelta:
// a = 7 - 2 = 5
a -= 2;

// b++ aumenta b en 1.
//
// Primera vuelta:
// b = 8
//
// Segunda vuelta:
// b = 9
b++;
}

// Valores finales:
//
// a = 5
// b = 9
// c = 6

// oper:
//
// a + b > 10 || c < 6
//
// a + b = 5 + 9 = 14
//
// 14 > 10 es true.
//
// c < 6:
// 6 < 6 es false.
//
// true || false = true.
boolean oper = a + b > 10 || c < 6;

// suma:
//
// a + b + c
//
// 5 + 9 + 6 = 20
int suma = a + b + c;

// mensaje:
//
// Si suma % 2 == 0, devuelve "PAR".
// Si no, devuelve "IMPAR".
//
// 20 % 2 = 0
//
// Entonces mensaje = "PAR".
String mensaje = (suma % 2 == 0) ? "PAR" : "IMPAR";

System.out.println(oper);

```



```

System.out.println(suma);
System.out.println(mensaje);
System.out.println("variable a es: " + a);
}
}

/*
VALORES FINALES:

a = 5
b = 9
c = 6
suma = 20
oper = true
mensaje = "PAR"

PREGUNTA:
¿Cuál afirmación es falsa?

a. El valor final de a es 5. Verdadera.
b. La suma impresa es 20. Verdadera.
c. oper es true. Verdadera.
d. mensaje es "IMPAR". Falsa.

RESPUESTA:
d. El valor que se imprime de mensaje es "IMPAR"

IDEA CLAVE:

El ternario:

condicion ? valorSiTrue : valorSiFalse

Como suma es par,
mensaje queda "PAR".
*/

```

Ejercicio 17 - Booleanos con ternario

CODIGO + EXPLICACION COMENTADA

```

public class Booleano {

    public static void main(String[] args) {

        boolean a = true;
        boolean b = false;

        // Expresión:
        //
        // c = (a && !b) ? !(a || b) : (a && b)
        //
        // Primero resolvemos la condición:
        //
        // a && !b
        //
        // a = true
        // b = false
        // !b = true
        //
        // true && true = true
        //
        // Como la condición del ternario es true,
        // se ejecuta la parte después del ?
        //
        // !(a || b)
        //
        // a || b
        // true || false = true
        //
        // !true = false
        boolean c = (a && !b) ? !(a || b) : (a && b);

        // Imprime lo contrario de c.
        //
        // c = false
        //
        // !c = true
        System.out.println(!c);
    }
}

```

```
/*  
PREGUNTA:  
¿Cuál es el contenido de c al final?  
  
c = false  
  
RESPUESTA:  
b. false  
  
OJO:  
El programa imprime !c, o sea true.  
Pero la pregunta pregunta por el contenido de c,  
no por lo que imprime.  
  
IDEA CLAVE:  
  
c vale false.  
!c vale true.  
  
Una cosa es la variable.  
Otra cosa es lo que imprimís.  
Trampa fina.  
*/
```

BLOQUE 4 - POO, clases y objetos

Ejercicio 18 - Objeto o instancia

CODIGO + EXPLICACION COMENTADA

```
public class Auto {  
  
    String marca;  
    String modelo;  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
  
        // Auto es la clase.  
        //  
        // a1 es una variable que referencia un objeto.  
        //  
        // new Auto() crea una instancia de Auto.  
        Auto a1 = new Auto();  
    }  
}  
  
/*  
RESPUESTA:  
d. Un objeto o instancia de Auto.  
  
EXPLICACIÓN:  
  
Auto:  
es la clase.  
  
a1:  
es la variable/referencia.  
  
new Auto():  
crea el objeto.  
  
Entonces:  
  
Auto a1 = new Auto();  
  
significa:  
crear un objeto de tipo Auto y guardarlo en la variable a1.  
  
IDEA CLAVE:  
  
Clase:  
molde.  
  
Objeto:  
cosa creada desde el molde.  
  
Auto es el plano.  
a1 es el auto fabricado.  
*/
```

Ejercicio 19 - Clase Libro completa

CODIGO + EXPLICACION COMENTADA

```
public class Libro {  
  
    private String titulo;  
    private String autor;  
    private int cantidadPaginas;  
  
    public Libro(String titulo, String autor, int cantidadPaginas) {  
        this.titulo = titulo;  
        this.autor = autor;  
        this.cantidadPaginas = cantidadPaginas;  
    }  
  
    public Libro(String titulo, String autor) {  
        this.titulo = titulo;  
        this.autor = autor;  
        this.cantidadPaginas = 0;  
    }  
}
```

```

}

public Libro() {
    this.titulo = "";
    this.autor = "";
    this.cantidadPaginas = 0;
}

public String getTitulo() {
    return titulo;
}

public void setTitulo(String titulo) {
    this.titulo = titulo;
}

public String getAutor() {
    return autor;
}

public void setAutor(String autor) {
    this.autor = autor;
}

public int getCantidadPaginas() {
    return cantidadPaginas;
}

public void setCantidadPaginas(int cantidadPaginas) {
    this.cantidadPaginas = cantidadPaginas;
}

public String mostrarInformacion() {

    // Devuelve los datos del libro en formato texto.
    return "Titulo: " + titulo +
        ", Autor: " + autor +
        ", Cantidad de paginas: " + cantidadPaginas;
}

public boolean esLargo() {

    // Devuelve true si el libro tiene más de 300 páginas.
    //
    // Si tiene 300 o menos, devuelve false.
    return cantidadPaginas > 300;
}
}

/*
EJEMPLO:

Libro libro = new Libro("Harry Potter", "J.K. Rowling", 223);

libro.esLargo();

223 > 300 es false.

IDEA CLAVE:

Constructores:
permiten crear objetos de distintas formas.

Getters:
devuelven datos.

Setters:
modifican datos.

mostrarInformacion:
devuelve String.

esLargo:
devuelve boolean.

El método esLargo() no imprime.
Solo responde true o false.
*/

```

Ejercicio 20 - Clase abstracta

CODIGO + EXPLICACION COMENTADA

```
abstract class Figura {  
  
    // Una clase abstracta puede tener atributos.  
    private String nombre;  
  
    // Puede tener constructor.  
    public Figura(String nombre) {  
        this.nombre = nombre;  
    }  
  
    // Puede tener métodos normales.  
    public String getNombre() {  
        return nombre;  
    }  
  
    // Puede tener métodos abstractos.  
    public abstract double calcularArea();  
}
```

/*
RESPUESTA CORRECTA:
c. Una clase que no se puede instanciar directamente.

EXPLICACIÓN:

Esto NO se puede:

```
Figura f = new Figura("algo");
```

Porque Figura es abstracta.

Pero sí se puede:

```
class Circulo extends Figura {  
    public Circulo() {  
        super("Círculo");  
    }  
  
    public double calcularArea() {  
        return 0;  
    }  
}
```

IDEA CLAVE:

Una clase abstracta:

- no se puede instanciar directamente
- puede tener atributos
- puede tener constructores
- puede tener métodos normales
- puede tener métodos abstractos

No está obligada a tener todos sus métodos abstractos.
Esa opción es falsa.
*/

Ejercicio 21 - Clase abstracta Auto y polimorfismo

CODIGO + EXPLICACION COMENTADA

```
// Clase abstracta.  
// No se puede instanciar directamente:  
//  
// Auto a = new Auto(); // ERROR  
//  
// Sirve como clase base para otros tipos de autos.  
abstract class Auto {  
  
    // Atributo para guardar el tipo de auto.  
    String tipo;  
  
    public Auto(String tipo) {  
        this.tipo = tipo;  
    }  
  
    // Método abstracto.  
    //  
    // No tiene cuerpo.
```

```

// Cada clase hija debe implementarlo.
public abstract double calcularConsumo();
}

class AutoNafta extends Auto {

private double kilometros;
private double costoPorKm;

public AutoNafta(double kilometros, double costoPorKm) {

// Llama al constructor de Auto.
super("Nafta");

this.kilometros = kilometros;
this.costoPorKm = costoPorKm;
}

@Override
public double calcularConsumo() {

// Nafta no tiene descuento.
//
// Fórmula:
// kilometros * costo por kilómetro
return kilometros * costoPorKm;
}
}

class AutoHibrido extends Auto {

private double kilometros;
private double costoPorKm;

public AutoHibrido(double kilometros, double costoPorKm) {
super("Hibrido");
this.kilometros = kilometros;
this.costoPorKm = costoPorKm;
}

@Override
public double calcularConsumo() {

// Hibrido tiene 30% de ahorro.
//
// Si ahorra 30%, paga el 70%.
//
// Por eso multiplicamos por 0.70.
return kilometros * costoPorKm * 0.70;
}
}

class AutoElectrico extends Auto {

private double kilometros;
private double costoPorKm;

public AutoElectrico(double kilometros, double costoPorKm) {
super("Electrico");
this.kilometros = kilometros;
this.costoPorKm = costoPorKm;
}

@Override
public double calcularConsumo() {

// Eléctrico tiene 50% de ahorro.
//
// Si ahorra 50%, paga la mitad.
//
// Por eso multiplicamos por 0.50.
return kilometros * costoPorKm * 0.50;
}
}

class Mostrar {

public static void mostrarDatos(java.util.List<Auto> lista) {

// Acumulador del consumo total.
double total = 0;

```

```

// Recorremos la lista de autos.
//
// Aunque la lista sea de tipo Auto,
// adentro puede haber AutoNafta, AutoHibrido o AutoElectrico.
for (Auto auto : lista) {

// getClass().getSimpleName() devuelve el nombre real de la clase.
//
// Ejemplo:
// AutoNafta
// AutoHibrido
// AutoElectrico
System.out.println(auto.getClass().getSimpleName());

// Polimorfismo:
//
// Java ejecuta calcularConsumo()
// según el tipo real del objeto.
//
// Si es AutoNafta, usa calcularConsumo() de AutoNafta.
// Si es AutoHibrido, usa calcularConsumo() de AutoHibrido.
// Si es AutoElectrico, usa calcularConsumo() de AutoElectrico.
total += auto.calcularConsumo();
}

// Imprime la suma total de consumos.
System.out.println(total);
}
}

/*
EJEMPLO:

100 km, costo 12

AutoNafta:
100 * 12 = 1200

AutoHibrido:
100 * 12 * 0.70 = 840

AutoElectrico:
100 * 12 * 0.50 = 600

Total:
1200 + 840 + 600 = 2640

SALIDA:
AutoNafta
AutoHibrido
AutoElectrico
2640.0

IDEA CLAVE:

Clase abstracta:
sirve como molde incompleto.

Método abstracto:
obliga a las clases hijas a implementarlo.

Polimorfismo:
Auto puede apuntar a objetos AutoNafta, AutoHibrido o AutoElectrico.

La magia real está acá:

total += auto.calcularConsumo();

Parece un solo método,
pero Java decide cuál ejecutar según el objeto real.
*/

```

Ejercicio 22 - Cuentas bancarias

CODIGO + EXPLICACION COMENTADA

```

class CuentaBancaria {

private String titular;
private double saldo;

```

```

public CuentaBancaria(String titular, double saldo) {
    this.titular = titular;
    this.saldo = saldo;
}

public String getTitular() {
    return titular;
}

public void setTitular(String titular) {
    this.titular = titular;
}

public double getSaldo() {
    return saldo;
}

public void setSaldo(double saldo) {
    this.saldo = saldo;
}

public double calcularInteres(int meses) {

    // En la clase base no se define interés real.
    //
    // Las clases hijas lo sobrescriben.
    return 0;
}

public double calcularSaldoFinal(int meses) {

    // Saldo final:
    // saldo + interés generado.
    return saldo + calcularInteres(meses);
}
}

class CajaAhorro extends CuentaBancaria {

    public CajaAhorro(String titular, double saldo) {
        super(titular, saldo);
    }

    @Override
    public double calcularInteres(int meses) {

        // Caja de ahorro:
        // 2% mensual.
        //
        // 2% = 0.02
        return getSaldo() * 0.02 * meses;
    }
}

class PlazoFijo extends CuentaBancaria {

    public PlazoFijo(String titular, double saldo) {
        super(titular, saldo);
    }

    @Override
    public double calcularInteres(int meses) {

        // Plazo fijo:
        // 5% mensual.
        //
        // 5% = 0.05
        return getSaldo() * 0.05 * meses;
    }
}

/*
EJEMPLO:

CajaAhorro con saldo 1000 por 3 meses:

interés = 1000 * 0.02 * 3
interés = 60

saldo final = 1000 + 60

```



```
saldo final = 1060
```

```
PlazoFijo con saldo 1000 por 3 meses:
```

```
interés = 1000 * 0.05 * 3
```

```
interés = 150
```

```
saldo final = 1150
```

```
IDEA CLAVE:
```

```
CuentaBancaria es la clase padre.
```

```
CajaAhorro y PlazoFijo heredan de CuentaBancaria.
```

```
Ambas sobrescriben calcularInteres().
```

```
Polimorfismo:
```

```
cada cuenta calcula el interés según su tipo real.
```

```
*/
```

BLOQUE 5 - Sobrecarga

Ejercicio 23 - Sobrecarga con short

CODIGO + EXPLICACION COMENTADA

```
public class Mostrar {

    public void mostrar(int a, int b) {

        // Método que recibe dos int.
        System.out.println("Metodo int: " + (a + b));
    }

    public void mostrar(double a, double b) {

        // Método que recibe dos double.
        System.out.println("Metodo double: " + (a + b));
    }

    public void mostrar(byte a, byte b) {

        // Método que recibe dos byte.
        System.out.println("Metodo byte: " + (a + b));
    }
}

public class TestMostrar {

    public static void main(String[] args) {

        Mostrar m = new Mostrar();

        // Llamamos al método con dos short.
        //
        // No existe un método:
        //
        // mostrar(short, short)
        //
        // Entonces Java busca una conversión válida.
        //
        // short puede promocionarse automáticamente a int.
        //
        // Por eso Java elige:
        // mostrar(int a, int b)
        m.mostrar((short) 5, (short) 7);
    }
}

/*
CÁLCULO:

5 + 7 = 12

Método elegido:
mostrar(int, int)

SALIDA:
Metodo int: 12

RESPUESTA:
a. Metodo int: 12

IDEA CLAVE:

Si no hay método exacto para short,
Java promociona short a int.

No elige byte,
porque short no se convierte automáticamente a byte.
Eso podría perder información.

Orden típico de promoción numérica:

byte -> short -> int -> long -> float -> double

Pero en operaciones y sobrecarga,
int suele ser el destino natural.
*/
```

Ejercicio 24 - Sobrecarga válida

CODIGO + EXPLICACION COMENTADA

```
public class Sobrecarga {  
  
    public int sumar(int a, int b) {  
        return a + b;  
    }  
  
    public double sumar(double a, double b) {  
        return a + b;  
    }  
}  
  
/*  
RESPUESTA CORRECTA:  
c. sumar(int a, int b) y sumar(double a, double b)  
  
POR QUÉ:  
  
Sobrecarga válida:  
mismo nombre, distintos parámetros.  
  
Acá ambos métodos se llaman sumar,  
pero uno recibe int,int  
y el otro recibe double,double.  
  
NO es sobrecarga:  
  
sumar(int a, int b)  
sumar(int g, int h)  
  
Aunque cambien los nombres de las variables,  
para Java la firma es igual.  
  
Java mira tipos, no nombres de parámetros.  
  
IDEA CLAVE:  
  
Firma:  
nombre + tipos de parámetros + orden  
  
No cuenta:  
nombre interno de las variables.  
*/
```

BLOQUE 6 - Colecciones

Ejercicio 25 - HashMap

CODIGO + EXPLICACION COMENTADA

```
import java.util.HashMap;
import java.util.Map;

public class PruebaMap {

    public static void main(String[] args) {

        Map<String, Integer> stock = new HashMap<>();

        // Agregamos tres claves.
        stock.put("Lapiz", 10);
        stock.put("Cuaderno", 5);
        stock.put("Goma", 8);

        // Actualizamos Lapiz.
        //
        // stock.get("Lapiz") devuelve 10.
        //
        // 10 + 5 = 15.
        //
        // Como "Lapiz" ya existe,
        // put reemplaza el valor anterior.
        stock.put("Lapiz", stock.get("Lapiz") + 5);

        // Eliminamos Cuaderno.
        stock.remove("Cuaderno");

        // Agregamos Regla con valor 7.
        stock.put("Regla", 7);

        // Actualizamos Goma.
        //
        // stock.get("Goma") devuelve 8.
        //
        // 8 - 3 = 5.
        stock.put("Goma", stock.get("Goma") - 3);

        System.out.println(stock);
    }
}

/*
ESTADO PASO A PASO:

Inicio:
{}

Después:
Lapiz = 10
Cuaderno = 5
Goma = 8

Luego:
Lapiz = 15

Luego:
Cuaderno se elimina.

Luego:
Regla = 7

Luego:
Goma = 5

ESTADO FINAL:

Lapiz = 15
Goma = 5
Regla = 7

Cantidad de elementos:
3

RESPUESTA:
```

d. El Map contiene 3 elementos, "Lapiz" queda con valor 15 y "Cuaderno" ya no está.

IDEA CLAVE:

```
put(clave, valor):
agrega si la clave no existe.
reemplaza si la clave ya existe.

remove(clave):
elimina esa clave.

get(clave):
obtiene el valor asociado a esa clave.
*/
```

Ejercicio 26 - LinkedHashSet

CODIGO + EXPLICACION COMENTADA

```
import java.util.LinkedHashSet;
import java.util.Set;

public class PrincipalSet {

    public static void main(String[] args) {

        // LinkedHashSet es un conjunto.
        //
        // Características:
        // 1) No permite duplicados.
        // 2) Mantiene el orden de inserción.
        Set<Integer> numeros = new LinkedHashSet<>();

        numeros.add(10); // Se agrega.
        numeros.add(20); // Se agrega.

        // 10 ya existe.
        // No se agrega duplicado.
        numeros.add(10);

        numeros.add(30); // Se agrega.

        // 20 ya existe.
        // No se agrega duplicado.
        numeros.add(20);

        // Como LinkedHashSet mantiene orden de inserción,
        // queda:
        //
        // 10, 20, 30
        System.out.println(numeros);
    }
}

/*
SALIDA:
[10, 20, 30]

RESPUESTA:
d. [10, 20, 30]

IDEA CLAVE:

HashSet:
no duplicados, orden no garantizado.

LinkedHashSet:
no duplicados, mantiene orden de inserción.

TreeSet:
no duplicados, ordena naturalmente.

Acá es LinkedHashSet,
por eso queda en el orden:
10, 20, 30.
*/
```

Ejercicio 27 - Queue con offer y poll

CODIGO + EXPLICACION COMENTADA

```
import java.util.LinkedList;
import java.util.Queue;

public class PrincipalCola {

    public static void main(String[] args) {

        // Queue representa una cola.
        //
        // Funciona como:
        // primero en entrar, primero en salir.
        //
        // FIFO:
        // First In, First Out.
        Queue<String> nombres = new LinkedList<>();

        // offer agrega al final de la cola.
        nombres.offer("Antonio");
        nombres.offer("Carmen");
        nombres.offer("Mariana");
        nombres.offer("Dante");

        // Estado:
        // [Antonio, Carmen, Mariana, Dante]

        // poll elimina y devuelve el primer elemento.
        //
        // Sale Antonio.
        nombres.poll();

        // Estado:
        // [Carmen, Mariana, Dante]

        // Sale Carmen.
        nombres.poll();

        // Estado:
        // [Mariana, Dante]

        // Agregamos Oscar al final.
        nombres.offer("Oscar");

        // Estado final:
        // [Mariana, Dante, Oscar]
        System.out.println(nombres);
    }
}

/*
SALIDA:
[Mariana, Dante, Oscar]

RESPUESTA:
a. [Mariana, Dante, Oscar]

IDEA CLAVE:

offer()
agrega al final.

poll()
saca el primero.

Queue es una fila:
el primero que llegó es el primero que sale.

Como en la panadería, pero sin ticket número 47.
*/
```

Ejercicio 28 - CRUD de reservas con HashMap

CODIGO + EXPLICACION COMENTADA

```
class Reserva {

    private int codigo;
    private String titular;
    private int noches;
    private double precioPorNoche;
```

```

public Reserva(int codigo, String titular, int noches, double precioPorNoche) {
    this.codigo = codigo;
    this.titular = titular;
    this.noches = noches;
    this.precioPorNoche = precioPorNoche;
}

public int getCodigo() {
    return codigo;
}

public double calcularTotal() {
    // Total de la reserva:
    // noches * precio por noche.
    return noches * precioPorNoche;
}

@Override
public String toString() {
    // Formato exacto para mostrar la reserva.
    return "Reserva{codigo=" + codigo +
        ", titular='" + titular + '\'' +
        ", noches=" + noches +
        ", precioPorNoche=" + precioPorNoche +
        ", total=" + calcularTotal() +
        '}';
}

interface ICrudReservas {

    void agregar(Reserva reserva);

    void modificar(Reserva reserva);

    void eliminar(int codigo);

    void listar();
}

class CrudReservasImpl implements ICrudReservas {

    // HashMap:
    //
    // Clave: Integer -> código de la reserva.
    // Valor: Reserva -> objeto completo.
    private java.util.HashMap<Integer, Reserva> reservas = new java.util.HashMap<>();

    @Override
    public void agregar(Reserva reserva) {
        // Agregamos la reserva usando su código como clave.
        //
        // Si ya existía una reserva con ese código,
        // put la reemplaza.
        reservas.put(reserva.getCodigo(), reserva);
    }

    @Override
    public void modificar(Reserva reserva) {
        // containsKey verifica si existe la clave.
        if (reservas.containsKey(reserva.getCodigo())) {
            // Si existe, reemplazamos la reserva anterior.
            reservas.put(reserva.getCodigo(), reserva);
        } else {
            // Si no existe, mostramos mensaje.
            System.out.println("No se pudo modificar a la reserva");
        }
    }

    @Override
    public void eliminar(int codigo) {
        // Verificamos si existe antes de eliminar.
    }
}

```

```

if (reservas.containsKey(codigo)) {

    reservas.remove(codigo);

} else {

    System.out.println("No se pudo eliminar a la reserva");
}
}

@Override
public void listar() {

    // Recorremos las claves del HashMap.
    //
    // keySet() devuelve el conjunto de claves.
    for (Integer clave : reservas.keySet()) {

        // Mostramos la clave y el objeto asociado.
        System.out.println("Clave: " + clave + " valor: " + reservas.get(clave));
    }
}

/*
IDEA CLAVE:

HashMap trabaja con pares:

clave -> valor

En este ejercicio:

codigo -> Reserva

agregar:
put(codigo, reserva)

modificar:
si existe, put reemplaza.
si no existe, mensaje.

eliminar:
si existe, remove.
si no existe, mensaje.

listar:
recorrer keySet() y mostrar clave + valor.

Esto es CRUD:

C -> Create -> agregar
R -> Read -> listar
U -> Update -> modificar
D -> Delete -> eliminar
*/

```


BLOQUE 7 - Excepciones

Ejercicio 29 - Afirmaciones sobre excepciones

CODIGO + EXPLICACION COMENTADA

```
public class ExcepcionesTeoria {  
  
    /*  
    Afirmación 1:  
    "El bloque try contiene el código que puede producir una excepción."  
  
    Correcta.  
  
    Ejemplo:  
    */  
  
    public void ejemploTry() {  
        try {  
            int x = 10 / 0; // Puede producir ArithmeticException  
        } catch (ArithmeticException e) {  
            System.out.println("Error aritmético");  
        }  
    }  
  
    /*  
    Afirmación 2:  
    "El bloque catch se ejecuta siempre, ocurra o no una excepción."  
  
    Incorrecta.  
  
    catch solo se ejecuta si ocurre una excepción compatible.  
    */  
  
    /*  
    Afirmación 3:  
    "El bloque finally se ejecuta siempre haya o no excepción."  
  
    Correcta.  
  
    finally se ejecuta normalmente siempre,  
    salvo casos extremos como apagar la JVM.  
    */  
  
    /*  
    Afirmación 4:  
    "La palabra throw se usa para lanzar una excepción."  
  
    Correcta.  
  
    Ejemplo:  
    throw new RuntimeException("Error");  
    */  
  
    /*  
    Afirmación 5:  
    "La palabra throws se usa dentro del bloque catch."  
  
    Incorrecta.  
  
    throws se usa en la firma del método.  
    */  
  
    public void metodo() throws Exception {  
        // throws avisa que este método puede lanzar una excepción.  
    }  
  
    /*  
    Afirmación 6:  
    "Un try no puede existir sin al menos un catch o un finally."  
  
    Correcta.  
  
    Esto está bien:  
  
    try {  
    } catch (Exception e) {  
    }  
  
    Esto también:
```

```
try {
} finally {
}
```

Pero try solo, sin catch ni finally, está mal.

```
*/
}
```

```
/*
AFIRMACIONES CORRECTAS:
1, 3, 4 y 6
```

```
RESPUESTA:
d. 1, 3, 4 y 6
```

IDEA CLAVE:

```
try:
intenta ejecutar código riesgoso.
```

```
catch:
captura errores si ocurren.
```

```
finally:
se ejecuta al final.
```

```
throw:
lanza una excepción.
```

```
throws:
declara que un método puede lanzar una excepción.
*/
```

Ejercicio 30 - Flujo con try, catch, finally

CODIGO + EXPLICACION COMENTADA

```
public class FlujoExcepcion {

    public static void main(String[] args) {

        // Bloque 1:
        // Código antes del try.
        System.out.println("Bloque 1");

        try {

            // Bloque 2:
            // Aquí ocurre una NumberFormatException.
            //
            // Ejemplo:
            Integer.parseInt("abc");
            System.out.println("Bloque 2");

        } catch (NullPointerException ex) {

            // Bloque 3:
            // Este catch NO se ejecuta,
            // porque la excepción no es NullPointerException.
            System.out.println("Bloque 3");

        } catch (Exception e) {

            // Bloque 4:
            // NumberFormatException hereda de Exception.
            //
            // Entonces este catch general sí la captura.
            System.out.println("Bloque 4");

        } finally {

            // Bloque 5:
            // finally se ejecuta siempre,
            // haya o no excepción.
            System.out.println("Bloque 5");
        }

        // Bloque 6:
        // Como la excepción fue capturada,
        // el programa continúa.
```

```

System.out.println("Bloque 6");
}
}

/*
SI OCURRE NumberFormatException EN BLOQUE 2:

Flujo:

Bloque 1
Bloque 2
Bloque 4
Bloque 5
Bloque 6

RESPUESTA:
b. Bloque 1 -> Bloque 2 -> Bloque 4 -> Bloque 5 -> Bloque 6

IDEA CLAVE:

catch específico solo captura su tipo.

NumberFormatException no entra en NullPointerException.

Pero sí entra en Exception,
porque Exception es más general.

finally siempre pasa por caja.
*/

```

Ejercicio 31 - Excepción no controlada en try interno

CODIGO + EXPLICACION COMENTADA

```

public class PruebaExcepcion {

    public static void main(String[] args) {

        try {

            int[] numeros = {10, 20, 30};

            try {

                // El arreglo tiene 3 elementos.
                //
                // Índices válidos:
                // 0, 1, 2
                //
                // numeros[5] no existe.
                //
                // Esto produce ArrayIndexOutOfBoundsException.
                System.out.println(numeros[5]);

            } catch (ArithmeticException e) {

                // Este catch solo captura errores aritméticos.
                //
                // Ejemplo:
                // división entre cero.
                //
                // Pero el error real es de índice fuera de rango.
                // Entonces este catch NO captura la excepción.
                System.out.println("Error aritmético");
            }

            // Esta línea no se ejecuta,
            // porque la excepción no fue capturada en el try interno.
            System.out.println("Fin del bloque interno");

        } catch (NullPointerException e) {

            // Este catch externo solo captura NullPointerException.
            //
            // Pero el error real es ArrayIndexOutOfBoundsException.
            //
            // Entonces tampoco lo captura.
            System.out.println("Error de referencia nula");
        }

        // Esta línea tampoco se ejecuta,

```

```
// porque la excepción queda sin controlar y el programa se detiene.
System.out.println("Fin del programa");
}
}

/*
RESPUESTA:
c. Se produce una excepción no controlada y el programa se detiene.

IDEA CLAVE:

Una excepción solo se captura si existe un catch compatible.

Error real:
ArrayIndexOutOfBoundsException

Catch interno:
ArithmeticException

Catch externo:
NullPointerException

Ninguno sirve.

Resultado:
el programa se detiene.

El try no es un escudo mágico.
Si el catch no coincide, Java te deja el golpe entero.
*/
```

Ejercicio 32 - Afirmaciones incorrectas sobre excepciones

CODIGO + EXPLICACION COMENTADA

```
public class TeoriaExcepciones {

    /*
    1 - catch puede existir sin try.

    Incorrecto.
    Un catch necesita un try antes.
    */

    /*
    2 - finally puede usarse junto con try.

    Correcto.
    */

    /*
    3 - throw se usa para lanzar una excepción.

    Correcto.
    */

    /*
    4 - throws se usa en la firma del método.

    Correcto.
    */

    public void metodo() throws Exception {
        // Ejemplo de throws en firma.
    }

    /*
    5 - El bloque finally se ejecuta únicamente si ocurre excepción.

    Incorrecto.
    finally se ejecuta haya o no excepción.
    */

    /*
    6 - Un catch puede capturar una excepción del tipo indicado.

    Correcto.
    */
}
```

AFIRMACIONES INCORRECTAS:

1 y 5

RESPUESTA:

c. 1 y 5

IDEA CLAVE:

catch sin try:
no existe.

finally:
no depende de que haya error.
Se ejecuta igual.
*/

Ejercicio 33 - Flujo sin excepción

CODIGO + EXPLICACION COMENTADA

```
public class FlujoSinExcepcion {

    public static void main(String[] args) {

        // Bloque 1:
        // Código antes del try.
        System.out.println("Bloque 1");

        try {

            // Bloque 2:
            // Código que se intenta ejecutar.
            //
            // En este caso, la pregunta dice que NO ocurre excepción.
            System.out.println("Bloque 2");

        } catch (Exception e) {

            // Bloque 4:
            // No se ejecuta porque no hubo excepción.
            System.out.println("Bloque 4");

        } finally {

            // Bloque 5:
            // Se ejecuta siempre,
            // haya o no excepción.
            System.out.println("Bloque 5");
        }

        // Bloque 6:
        // Después del try/catch/finally,
        // el programa continúa.
        System.out.println("Bloque 6");
    }

    /*
    SI NO OCURRE EXCEPCIÓN:

    Flujo:

    Bloque 1
    Bloque 2
    Bloque 5
    Bloque 6

    RESPUESTA:
    d. Bloque 1 -> Bloque 2 -> Bloque 5 -> Bloque 6

    IDEA CLAVE:

    Si no hay excepción:
    catch no se ejecuta.

    finally sí se ejecuta.

    Después continúa el programa.
    */
```

Nota final - ejercicios repetidos no duplicados

CODIGO + EXPLICACION COMENTADA

Ejercicios que NO repetí porque ya estaban explicados
/*

No repetí estos porque ya los expliqué antes en el parcial unificado:

- Interfaces e implements.
- Función vs procedimiento.
- Clase Libro básica.
- Herencia y polimorfismo Animal/Perro/Gato.
- Excepciones generales try/catch/finally.
- HashSet sin duplicados.
- LinkedList.
- Métodos void.
- Clases abstractas en general.
- Arreglos y matrices básicos.

Los únicos nuevos o con una trampa distinta
sí quedaron explicados arriba.

Con esto quedan pasados en limpio los dos PDF:

- 1) Parcial unificado.
- 2) Pre Evaluación FEIP 2026.

Mapa final de patrones:

Arrays:
recorrer con for.

Matrices:
doble for.

Listas:
add, remove, set, get.

Queue:
offer y poll.

Set:
no duplicados.

Map:
clave -> valor.

String:
equals, substring, charAt.

P00:
clase, objeto, constructor, getters, setters.

Herencia:
extends.

Interfaz:
implements.

Polimorfismo:
se ejecuta el método del objeto real.

Excepciones:
try, catch, finally.

Acumuladores:
suma, contador, promedio, factorial.

La clave para salvar este tipo de parcial:
no memorizar 40 códigos sueltos,
sino reconocer el patrón que se repite.
*/